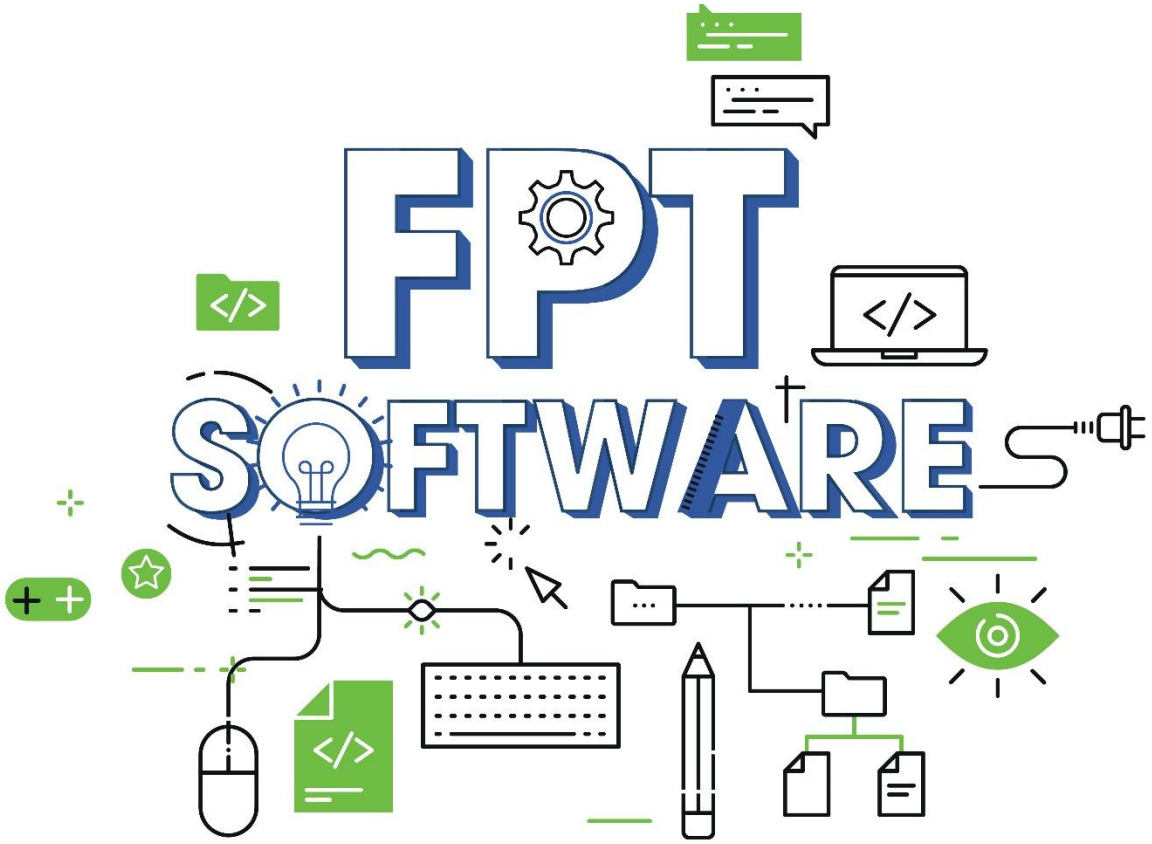
Basic Android

Fragment



Agenda

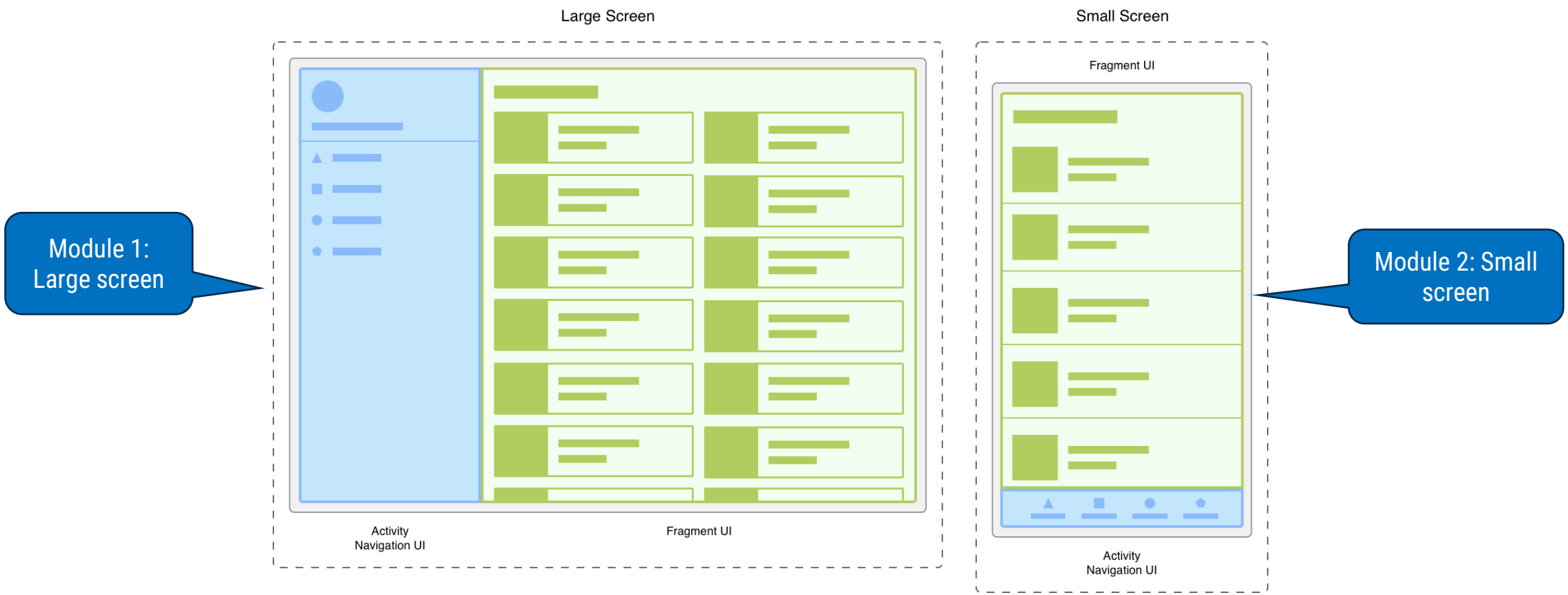
1. Overview
2. Create a fragment
3. Fragment manager
4. Navigate between fragments using animations
5. Fragment lifecycle
6. Saving state with fragments
7. Communicate with fragments
8. Fragment transactions

Overview

What is Fragments ?

- Represents a reusable portion of your app's UI.
- Defines and manages its own layout, has its own lifecycle, and can handle its own input events.
- Fragments can't live on their own. They must be *hosted* by an activity or another fragment.

Modularity


Large Screen
Small Screen
Fragment UI
Module 1:
Large screen
Module 2: Small
screen
Activity
Navigation UI
Fragment UI
Activity
Navigation UI

Modularity

- Fragments introduce modularity and reusability into your activity's UI by letting you divide the UI into discrete chunks.
- Dividing your UI into fragments makes it easier to modify your activity's appearance at runtime. While your activity is in the **STARTED** lifecycle state or higher, fragments can be added, replaced, or removed.

Create a fragment

Create a fragment

- A fragment represents a modular portion of the user interface within an activity.
- A fragment has its own lifecycle, receives its own input events, and you can add or remove fragments while the containing activity is running.

Follow next slides to describe how to create a fragment and include it into an activity

Setup environment

- Fragments require a dependency on the AndroidX Fragment library. You need to add the Google Maven repository to your project's settings.gradle file in order to include this dependency.

Groovy Kotlin

```
dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        ...
    }
}
```

Setup environment

- To include the AndroidX Fragment library to your project, add the following dependencies in your app's build.gradle file:

Groovy Kotlin

```
dependencies {
    val fragment_version = "1.5.7"

    // Java language implementation
    implementation("androidx.fragment:fragment:$fragment_version")
    // Kotlin
    implementation("androidx.fragment:fragment-ktx:$fragment_version")
}
```

Create a fragment class

- To create a fragment, extend the AndroidX `Fragment` class, and override its methods to insert your app logic.
- To create a minimal fragment that defines its own layout, provide your fragment's layout resource to the base constructor.

Kotlin Java

```kotlin
class ExampleFragment : Fragment(R.layout.example_fragment)
```

Add fragment to an activity via XML

- To declaratively add a fragment to your activity layout's XML, use a FragmentContainerView element.

```
<!-- res/layout/example_activity.xml -->
<androidx.fragment.app.FragmentContainerView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/fragment_container_view"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:name="com.example.ExampleFragment" />
```

Add fragment via XML

The android:name attribute specifies the class name of the Fragment to instantiate. When the activity's layout is inflated, the specified fragment is instantiated, onInflate() is called on the newly instantiated fragment, and a FragmentTransaction is created to add the fragment to the FragmentManager.

Add fragment to an activity programatically

- To programmatically add a fragment to your activity's layout, the layout should include a FragmentContainerView to serve as a fragment container.

```
<!-- res/layout/example_activity.xml -->
<androidx.fragment.app.FragmentContainerView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/fragment_container_view"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

- Unlike the XML approach, the android:name attribute isn't used on the FragmentContainerView here
- Instead, a FragmentTransaction is used to instantiate a fragment and add it to the activity's layout.

Add fragment to an activity programatically

- While your activity is running, you can make fragment transactions such as adding, removing, or replacing a fragment.

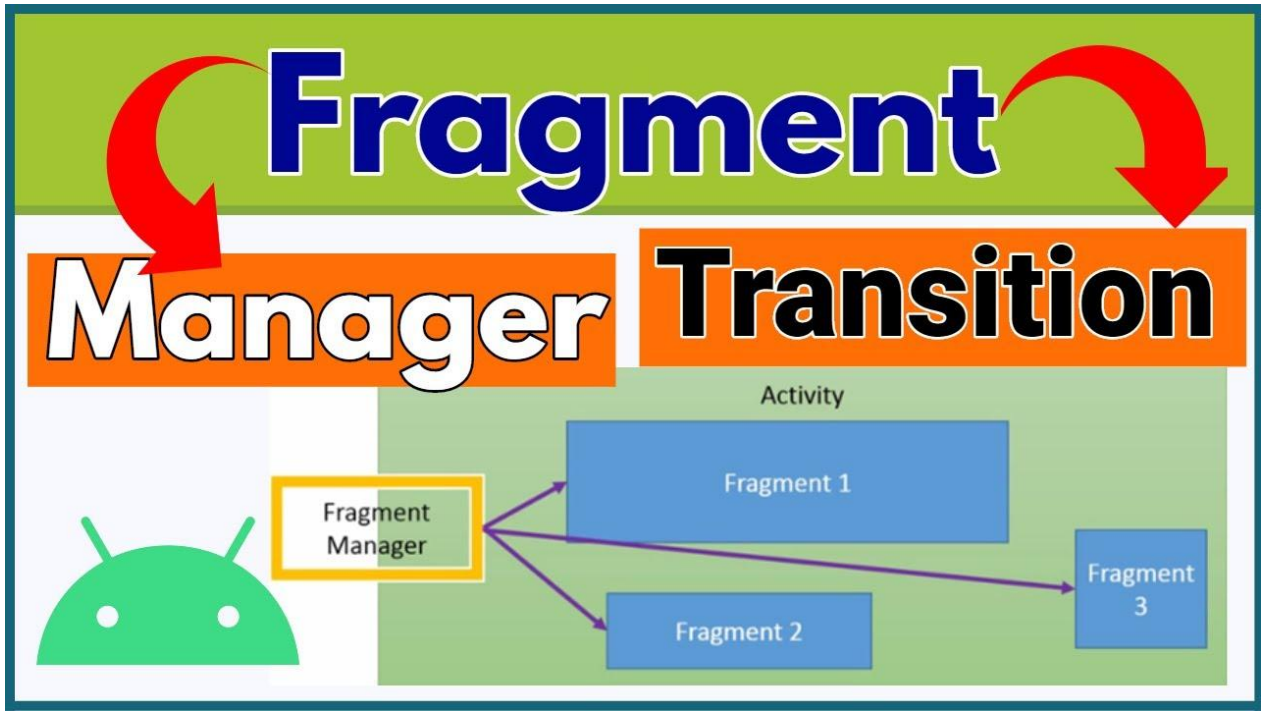
Kotlin Java

```kotlin
class ExampleActivity : AppCompatActivity(R.layout.example_activity) {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        if (savedInstanceState == null) {
            supportFragmentManager.commit {
                setReorderingAllowed(true)
                add<ExampleFragment>(R.id.fragment_container_view)
            }
        }
    }
}
```

Fragment manager

What's FragmentManager

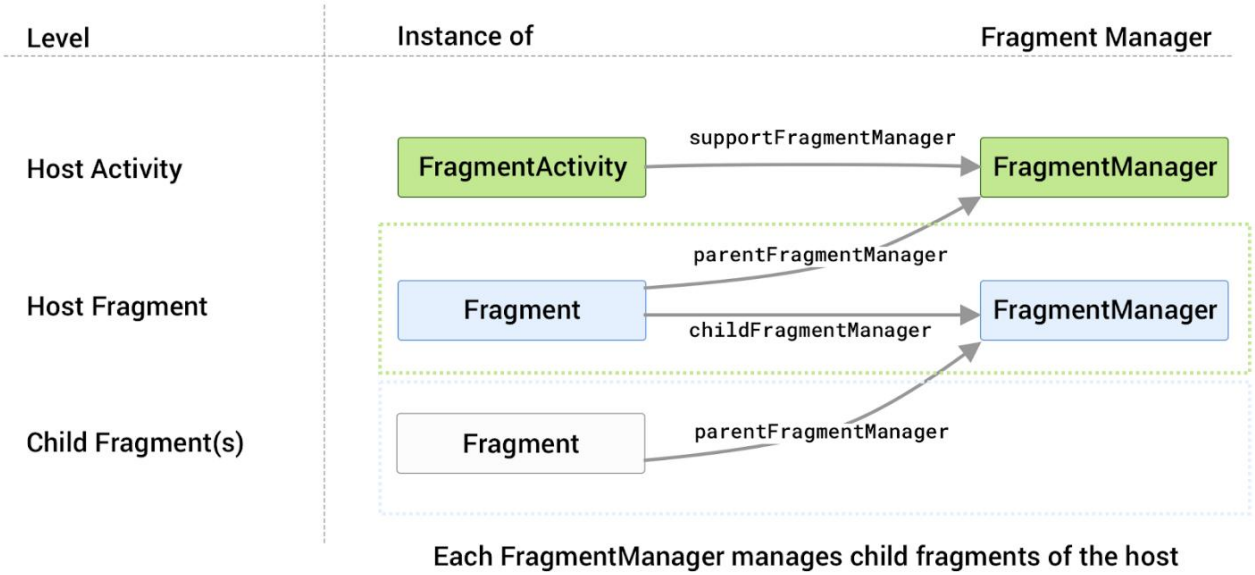
- `FragmentManager` is the class responsible for performing actions on your app's fragments, such as adding, removing, or replacing them and adding them to the back stack.



Access the FragmentManager

- Can access the FragmentManager from an activity or from a fragment.
- `FragmentActivity` and its subclasses, such as `AppCompatActivity`, have access to the FragmentManager through the `getSupportFragmentManager()` method.
- Fragments can host one or more child fragments. Inside a fragment, you can get a reference to the FragmentManager that manages the fragment's children through `getChildFragmentManager()`. If you need to access its host FragmentManager, you can use `getParentFragmentManager()`.

Access the FragmentManager



Two UI layout examples showing the relationships between fragments and their host activities.

Each host has its own FragmentManager associated with it that manages its child fragments.

Use the FragmentManager

- Each set of changes is committed together as a single unit called a `FragmentTransaction`.
- When the user taps the Back button on their device, or when you call `FragmentManager.popBackStack()`, the top-most fragment transaction pops off of the stack.
- When you call `addToBackStack()` on a transaction, the transaction can include any number of operations, such as adding multiple fragments or replacing fragments in multiple containers.

Find an existing fragment

- Use findFragmentById() to look up a fragment either by the given ID when inflated from XML or by the container ID when added in a FragmentTransaction

Kotlin Java

```kotlin
supportFragmentManager.commit {
    replace<ExampleFragment>(R.id.fragment_container)
    setReorderingAllowed(true)
    addToBackStack(null)
}
...
val fragment: ExampleFragment =
        supportFragmentManager.findFragmentById(R.id.fragment_container) as ExampleFragment
```

Find an existing fragment (2)

- Alternatively, you can assign a unique tag to a fragment and get a reference using `findFragmentByTag()` or assign a tag using the `android:tag` XML attribute on fragments.

Kotlin Java

```
supportFragmentManager.commit {
    replace<ExampleFragment>(R.id.fragment_container, "tag")
    setReorderingAllowed(true)
    addToBackStack(null)
}
...
val fragment: ExampleFragment =
        supportFragmentManager.findFragmentByTag("tag") as ExampleFragment
```

Support multiple back stacks

Kotlin Java

```kotlin
supportFragmentManager.commit {
  replace<ExampleFragment>(R.id.fragment_container)
  setReorderingAllowed(true)
  addToBackStack("replacement")
}
```

Add fragment to the back stack

Kotlin Java

```kotlin
supportFragmentManager.saveBackStack("replacement")
```

Save the state of fragment

Kotlin Java

```kotlin
supportFragmentManager.restoreBackStack("replacement")
```

Restore all of the saved fragment states

Fragment transactions

What's fragment transactions ?

- At runtime, a FragmentManager can add, remove, replace, and perform other actions with fragments in response to user interaction.
- Each set of fragment changes that you commit is called a *transaction*, and you can specify what to do inside the transaction using the APIs provided by the FragmentTransaction class.
- You can group multiple actions into a single transaction.
- You can save each transaction to a back stack managed by the FragmentManager and get an instance of FragmentTransaction from the FragmentManager by calling beginTransaction()

Adding and removing fragment

- To add a fragment to a FragmentManager, call `add()` on the transaction. The added fragment is moved to the **RESUMED** state.
- To remove a fragment from the host, call `remove()`, passing in a fragment instance that was retrieved from the fragment manager through `findFragmentById()` or `findFragmentByTag()`. The removed fragment is moved to the **DESTROYED** state.
- Use `replace()` to replace an existing fragment in a container with an instance of a new fragment class. Calling replace() is equivalent to calling remove() with a fragment in a container and adding a new fragment to that same container.

Commit is asynchronous

- Calling commit() doesn't perform the transaction immediately, the transaction is scheduled to run on the main UI thread as soon as it is able to do so.
- Call commitNow() to run the fragment transaction on your UI thread immediately, but it is incompatible with addToBackStack.
- Can execute all pending FragmentTransactions submitted by commit() calls that have not yet run by calling executePendingTransactions().

For the vast majority of use cases, commit() is all you need.

Navigate between fragments using animations

Set animations

Set animations (1)

- Animations can be defined in the res/anim directory.
- Animations can be defined as follows:

```xml
<!-- res/anim/slide_out.xml -->
<translate xmlns:android="http://schemas.android.com/apk/res/android"
    android:duration="@android:integer/config_shortAnimTime"
    android:interpolator="@android:anim/decelerate_interpolator"
    android:fromXDelta="0%"
    android:toXDelta="100%" />
```

```xml
<!-- res/anim/fade_in.xml -->
<alpha xmlns:android="http://schemas.android.com/apk/res/android"
    android:duration="@android:integer/config_shortAnimTime"
    android:interpolator="@android:anim/decelerate_interpolator"
    android:fromAlpha="0"
    android:toAlpha="1" />
```

Set animations (2)

- Once you've defined your animations, use them by calling `FragmentTransaction.setCustomAnimations()`, passing animation resources by their resource ID.

Kotlin Java

```kotlin
val fragment = FragmentB()
supportFragmentManager.commit {
    setCustomAnimations(
        enter = R.anim.slide_in,
        exit = R.anim.fade_out,
        popEnter = R.anim.fade_in,
        popExit = R.anim.slide_out
    )
    replace(R.id.fragment_container, fragment)
    addToBackStack(null)
}
```

Set transitions

- You can also use transitions to define enter and exit effects. These transitions can be defined in XML resource files.
- These transitions can be defined as follows:

```xml
<!-- res/transition/fade.xml -->
<fade xmlns:android="http://schemas.android.com/apk/res/android"
    android:duration="@android:integer/config_shortAnimTime"/>
```

```xml
<!-- res/transition/slide_right.xml -->
<slide xmlns:android="http://schemas.android.com/apk/res/android"
    android:duration="@android:integer/config_shortAnimTime"
    android:slideEdge="right" />
```

Set transitions (2)

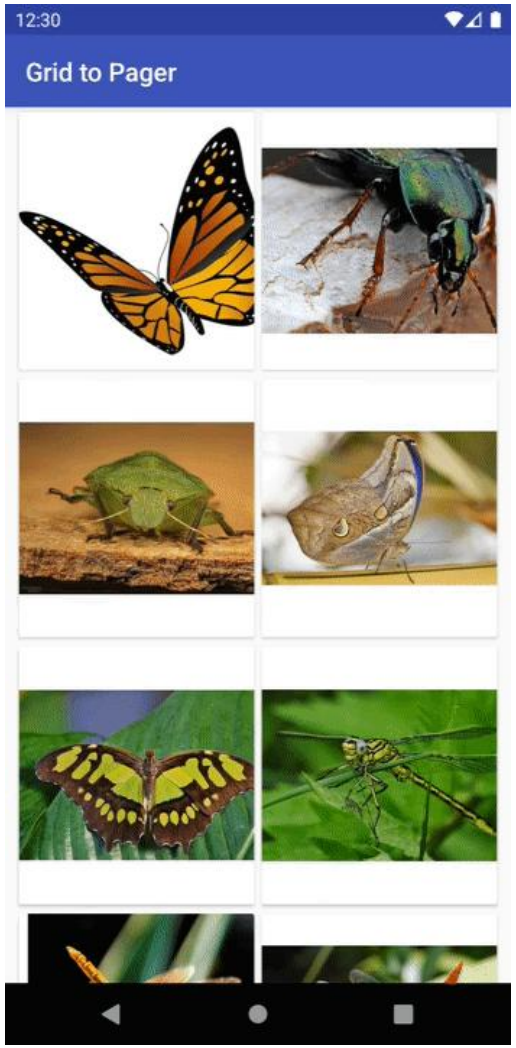
- Once you've defined your transitions, apply them by calling setEnterTransition() on the entering fragment and setExitTransition() on the exiting fragment

Kotlin Java

```kotlin
class FragmentA : Fragment() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        val inflater = TransitionInflater.from(requireContext())
        exitTransition = inflater.inflateTransition(R.transition.fade)
    }
}

class FragmentB : Fragment() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        val inflater = TransitionInflater.from(requireContext())
        enterTransition = inflater.inflateTransition(R.transition.slide_right)
    }
}
```

Use shared element transitions



Use shared element transitions (1)

- First, you must assign a unique transition name to each shared element view to allow the views to be mapped from one fragment to the next
- Set a transition name on shared elements in each fragment layout using ViewCompat.setTransitionName()

Kotlin Java

```kotlin
class FragmentA : Fragment() {
    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        ...
        val itemImageView = view.findViewById<ImageView>(R.id.item_image)
        ViewCompat.setTransitionName(itemImageView, "item_image")
    }
}

class FragmentB : Fragment() {
    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        ...
        val heroImageView = view.findViewById<ImageView>(R.id.hero_image)
        ViewCompat.setTransitionName(heroImageView, "hero_image")
    }
}
```

Use shared element transitions (2)

- Add each of your shared elements to your FragmentTransaction by calling `FragmentTransaction.addSharedElement()`

Kotlin Java

```
val fragment = FragmentB()
supportFragmentManager.commit {
    setCustomAnimations(...)
    addSharedElement(itemImageView, "hero_image")
    replace(R.id.fragment_container, fragment)
    addToBackStack(null)
}
```

Use shared element transitions (3)

- To specify how your shared elements transition from one fragment to the next, you must set an *enter* transition on the fragment being navigated to

Kotlin Java

```kotlin
class FragmentB : Fragment() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        sharedElementEnterTransition = TransitionInflater.from(requireContext())
            .inflateTransition(R.transition.shared_image)
    }
}
```

» The shared_image transition is defined as follows:

```xml
<!-- res/transition/shared_image.xml -->
<transitionSet>
    <changeImageTransform />
</transitionSet>
```

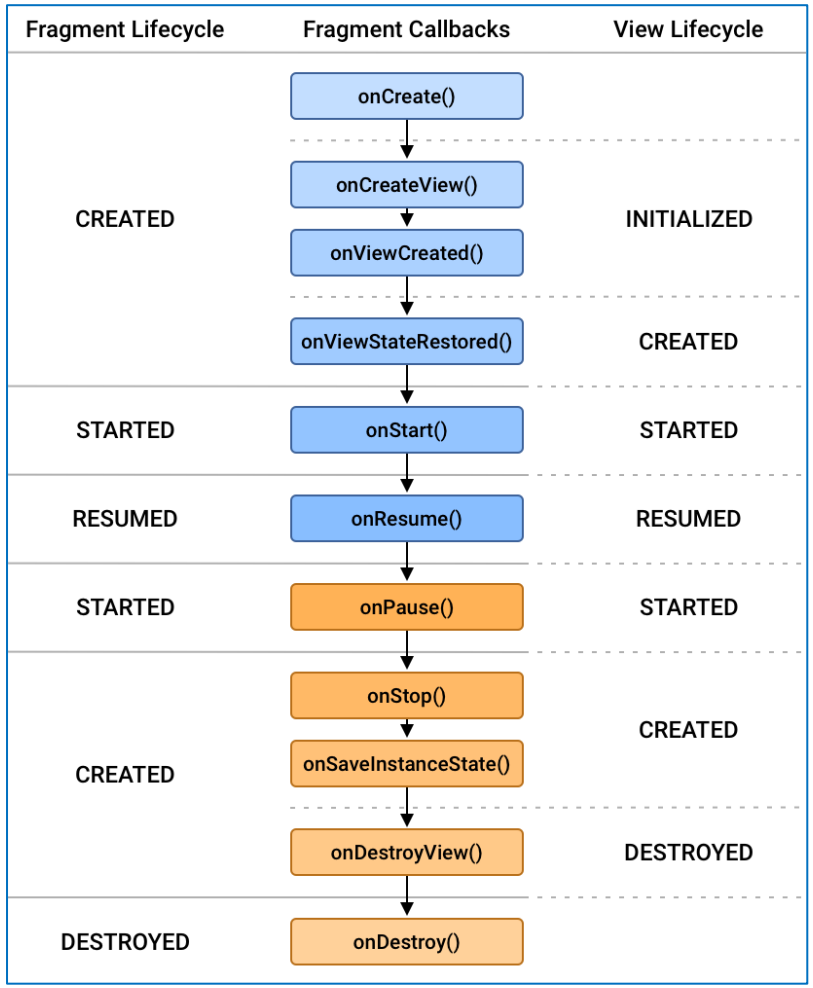
Fragment lifecycle

Fragment lifecycle

- To manage lifecycle, Fragment implements LifecycleOwner, exposing a Lifecycle object that you can access through the getLifecycle() method.
- Each possible Lifecycle state is represented in the Lifecycle.State enum.
  - INITIALIZED
  - CREATED
  - STARTED
  - RESUMED
  - DESTROYED

Fragment lifecycle

- When a fragment is added to the top of the back stack moves upward from **created** to **started** to **resumed**.
- When a fragment is popped off of the back stack, it moves downward from **resumed** to **stated** to **created** and finally **destroyed**.



Explaination for each state

- **Fragment created**: When fragment reaches the **created** state, it has been added to a FragmentManager and the onAttach() method has already been called.
- **Fragment created and View initialized:** This is the appropriate place to set up the initial state of your view.
- **Fragment and View created:** After the fragment's view has been created, the previous view state, if any, is restored, and the view's Lifecycle is then moved into the **created** state.
- **Fragment and View resumed:** When the fragment is visible, and the fragment is ready for user interaction. The fragment's Lifecycle moves to the **resumed** state, and the onResume() callback is invoked.

Explaination for each state (2)

- **Fragment and View started:** As the user begins to leave the fragment, and while the fragment is still visible. its view are moved back to the **started** state.
- **Fragment and View created:** Once the fragment is no longer visible, the Lifecycles for the fragment and for its view are moved into the **created** state and emit the `onStop()` event.
- **Fragment created and View destroyed:** After all of the exit `animations and transitions` have completed, and the fragment's view has been detached from the window.
- **Fragment destroyed:** If the fragment is removed, or if the FragmentManager is destroyed, the fragment's Lifecycle is moved into the **destroyed** state.

Saving state with fragments

Saving state with fragments

- Various Android system operations can affect the state of your fragment. To ensure the user's state is saved, the Android framework automatically saves and restores the fragments and the back stack.

Operation	Variables	View State	SavedState	NonConfig
Added to back stack	✓	✓	x	✓
Config Change	x	✓	✓	✓
Process Death/Recreation	x	✓	✓	✓*
Removed not added to back stack	x	x	x	x
Host finished	x	x	x	x

The operations that cause your fragment to lose state

View state

- Views are responsible for managing their own state.
- When a view accepts user input, it is the view's responsibility to save and restore that input to handle configuration changes.
- All Android framework-provided views have their own implementation of onSaveInstanceState() and onRestoreInstanceState()

Don't have to manage view state within the fragment.

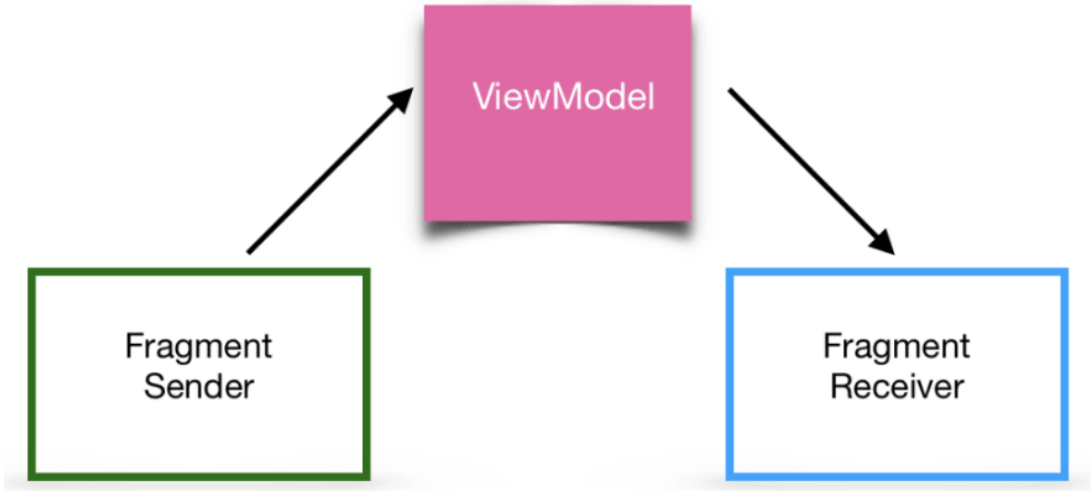
Saved state

- Fragment is responsible for managing small amounts of dynamic state that are integral to how the fragment functions.
- Can retain easily-serialized data using Fragment.onSaveInstanceState(Bundle).
- The data placed in the bundle is retained through configuration changes and process death and recreation and is available in fragment's onCreate(Bundle), onCreateView(LayoutInflater, ViewGroup, Bundle), and onViewCreated(View, Bundle) methods.

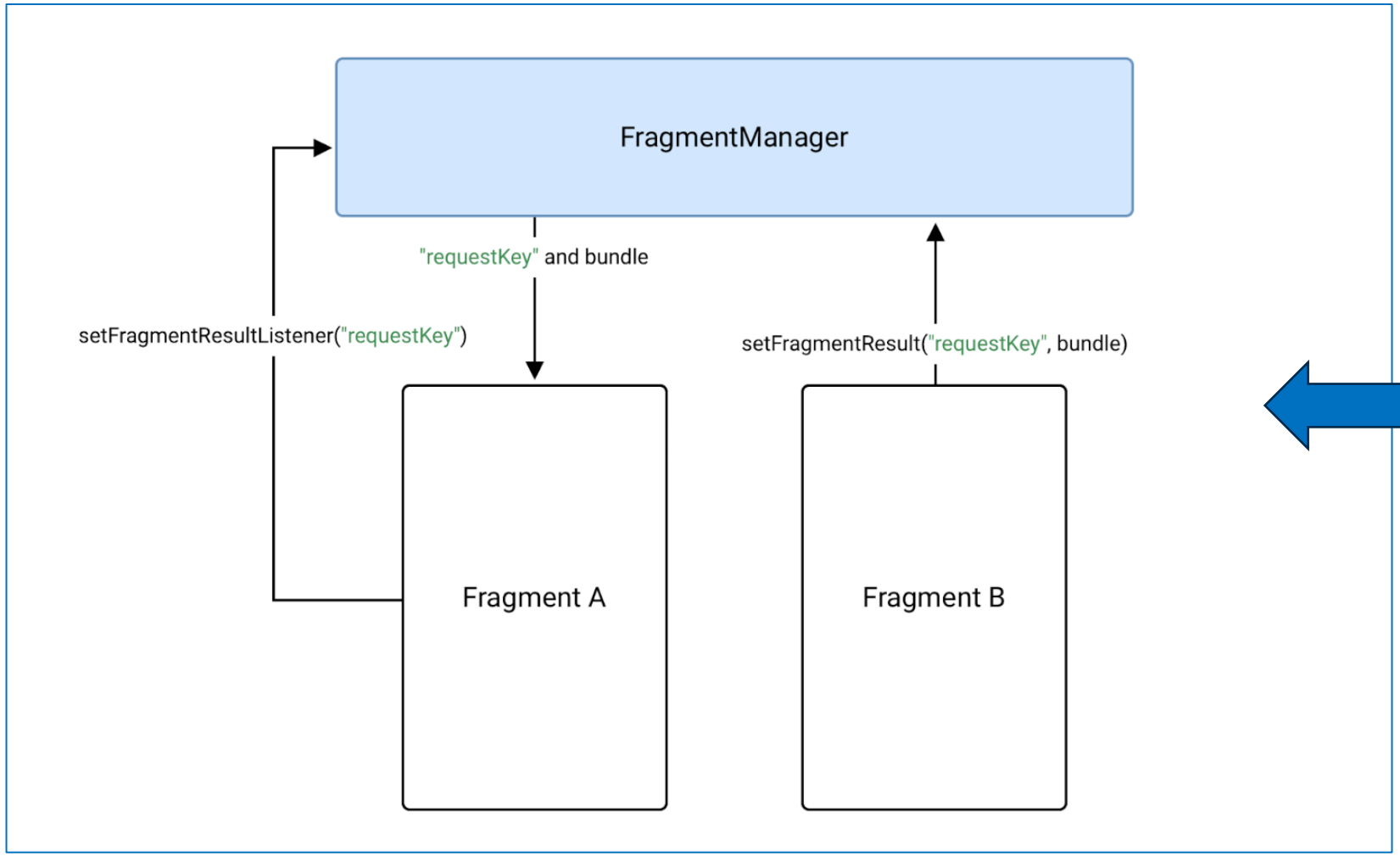
Communicate with fragments

Share data using ViewModel

- `ViewModel` is an ideal choice when you need to share data between multiple fragments or between fragments and their host activity. ViewModel objects store and manage UI data.

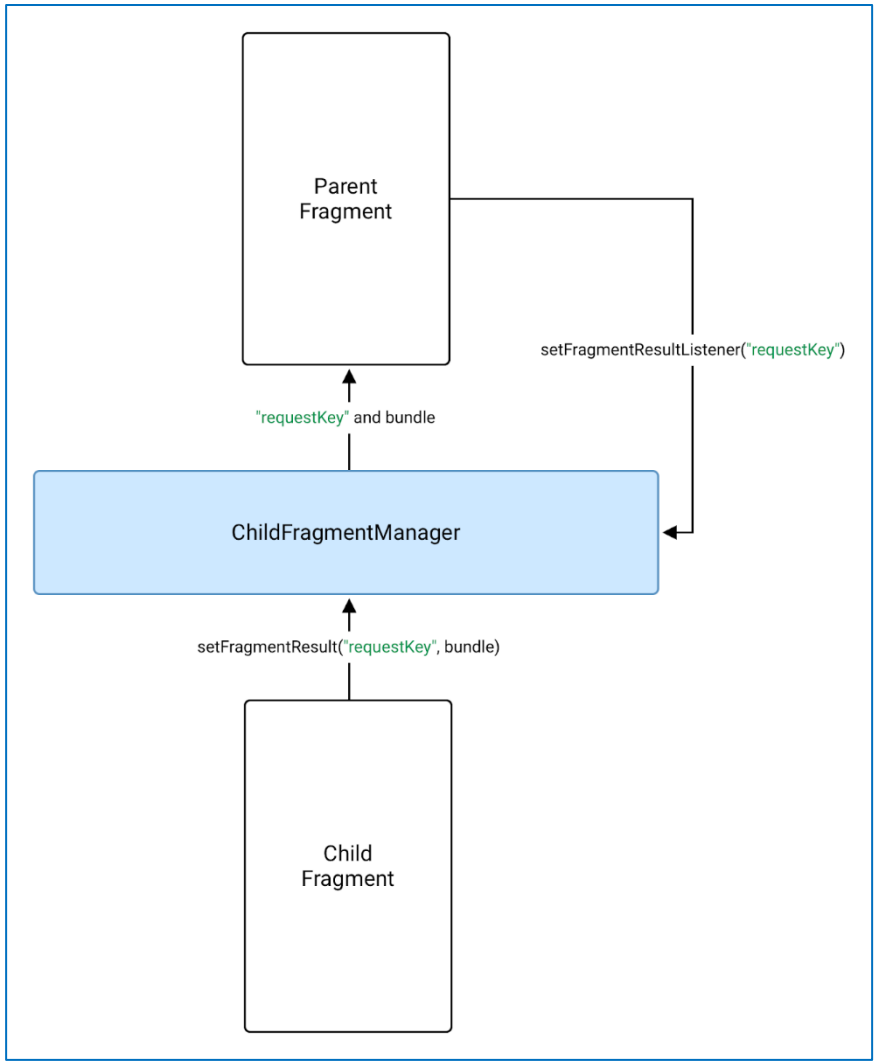


Get results using the Fragment result API



Pass results between fragments

Get results using the Fragment result API



Pass results between parent and child fragments

THANK YOU!